

Entregable 2: Bitácora de uso de IA generativa

Sistema de Gestión de Fincas e Inmuebles

Integrantes (Grupo 1)

- Fabio Perez Ruiz
- Maria Alejandra Ramos Naizir
- Karla Tom Cabarcas

Fecha	Herramienta utilizada	Prompt o pregunta realizada	Resumen de la respuesta recibida	Decisión del grupo	Justificación
02/05/26	ChatGPT	Estoy diseñando un sistema en Java para una inmobiliaria con clases como Inmueble, Edificio, Piso, Local, Inquilino, FacturaGasto y MovimientoBancario . ¿Cómo puedo mejorar el diseño aplicando herencia, evitar duplicidad y representar correctamente la relación entre edificios, pisos y locales?	Se analizó el modelo propuesto y se sugirió convertir Inmueble en una clase abstracta, además de crear una clase Alquiler para representar la relación entre inquilinos e inmuebles.	Aceptado	Se evitó la duplicación de atributos y se asignó al alquiler una responsabilidad propia dentro del modelo.
02/05/2026	ChatGPT	Revisa el diagrama UML de nuestra aplicación inmobiliaria y sugiere mejoras relacionadas con las relaciones entre clases, multiplicidades, métodos representados y uso de enumeraciones, procurando que el modelo sea consistente con los principios de Programación Orientada a Objetos.	Identificó mejoras relacionadas con la representación de métodos públicos, la incorporación de enumeraciones para atributos con valores limitados y ajustes en relaciones y multiplicidades entre las clases.	Aceptado	Se realizaron las correcciones sugeridas para obtener un diagrama más claro y consistente antes de comenzar la implementación del proyecto.
25/06/2026	Claude	Estamos desarrollando una aplicación de gestión inmobiliaria en Java utilizando Programación Orientada a Objetos. Necesitamos implementar validaciones para	Recomendó implementar excepciones personalizadas mediante clases como EdadInvalida, NumeroNegativo y CedulaIncorrecta,	Rechazado	El grupo consideró que, para el alcance del proyecto, una solución basada en múltiples excepciones personalizadas agregaba complejidad

		datos como la edad del inquilino, la cédula y valores numéricos (precio de alquiler, costos, importes, etc.). Considerando que el proyecto solo utiliza clases, herencia, clases abstractas y manejo básico de excepciones, ¿cuál sería la mejor estrategia para organizar estas validaciones? ¿Es preferible crear excepciones personalizadas para cada caso o centralizar las validaciones en una clase utilitaria? Justifica la recomendación.	permitiendo identificar y gestionar de forma independiente cada tipo de error.		innecesaria. Se optó por implementar una clase Validador con métodos estáticos para centralizar las verificaciones, obteniendo una solución más simple y fácil de mantener.
25/06/2026	Claude	Genera una propuesta de arquitectura para una aplicación de gestión inmobiliaria en Java. Organiza el proyecto en paquetes, define una jerarquía inicial de clases y proporciona un ejemplo de persistencia utilizando archivos .dat, siguiendo los principios de Programación Orientada a Objetos y manteniendo una separación adecuada de responsabilidades.	Propuso organizar el proyecto en paquetes como modelo, vista, servicio y repositorio, además de presentar una jerarquía inicial de clases y un ejemplo para implementar la persistencia mediante archivos .dat	Aceptado	La propuesta facilitó una organización más clara del proyecto, permitiendo separar la lógica del negocio, la interfaz, el almacenamiento y las entidades del sistema, lo que mejoró la mantenibilidad y escalabilidad del código.
27/06/2026	Claude	Implementa la lógica del método de búsqueda para los paneles de Facturas y Movimientos Bancarios en una aplicación Java. El método debe permitir filtrar los registros por fecha, por identificador o por ambos criterios,	Generó un método de búsqueda que filtraba correctamente los registros, pero solo devolvía resultados cuando se proporcionaban simultáneamente	Modificado	El grupo ajustó la implementación para permitir búsquedas utilizando únicamente la fecha, únicamente el identificador o ambos criterios al mismo

		garantizando una búsqueda flexible y eficiente sin duplicar código.	la fecha y el identificador.		tiempo, mejorando la flexibilidad y la experiencia de uso del sistema sin alterar la lógica principal propuesta por la IA.
27/06/2026	Gemini	En una aplicación de gestión inmobiliaria desarrollada en Java, ¿es recomendable implementar la opción de eliminar facturas y movimientos bancarios? Considera las buenas prácticas de los sistemas de información y el impacto que esta funcionalidad podría tener sobre la integridad y trazabilidad de los registros financieros.	Advirtió que en sistemas de información reales, los flujos bancarios no se borran por auditoría. Sugirió dejarlos de solo lectura o aplicar lógica de reversión contable.	Aceptado	Decidimos seguir el consejo y el enunciado del proyecto: dejar los paneles financieros como históricos de solo lectura, evitando la complejidad innecesaria de pagos reales.
28/06/2026	Claude, DeepSeek	Genera una propuesta de diseño visual para la pestaña de la aplicación inmobiliaria encargada de administrar los registros del sistema. La interfaz debe ser clara, intuitiva y organizada, incluyendo la distribución de botones, campos de búsqueda, tablas de información y acciones disponibles, manteniendo coherencia con el resto de la aplicación desarrollada en Java Swing	Propusieron una distribución visual de la pestaña, organizando los componentes mediante paneles, formularios, tablas y botones para facilitar la navegación y el acceso a las funciones principales.	Aceptado	El grupo tomó como base la distribución propuesta por ambas IA, pero realizó ajustes en la ubicación de algunos componentes, el orden de los botones y la organización de los paneles para adaptarla a las necesidades específicas del proyecto y mantener una interfaz uniforme en toda la aplicación.
29/06/2026	Claude	Estamos desarrollando la interfaz en Java Swing y ya seleccionamos una paleta de colores para la aplicación.	Propusieron aplicar la paleta elegida de forma consistente mediante constantes reutilizables y	Aceptado	Las recomendaciones facilitaron una apariencia homogénea en toda la aplicación sin

		Ayúdanos a implementar esos colores de manera uniforme en botones, paneles, tablas y formularios, procurando mantener una apariencia limpia y consistente en todas las ventanas.	mantener criterios uniformes para fondos, botones, tablas y componentes de la interfaz.		modificar la paleta de colores definida previamente por el grupo.
29/06/2026	Claude, DeepSeek	Estamos desarrollando una aplicación inmobiliaria en Java con persistencia en archivos .dat. ¿Cuál es la mejor estrategia para administrar los contadores de los identificadores (ID) de las diferentes entidades, garantizando que no se repitan al reiniciar la aplicación?	Claude recomendó almacenar los contadores de cada entidad en un archivo independiente, de manera que al iniciar la aplicación se recuperaran los últimos valores utilizados y se continuara la numeración sin generar identificadores duplicados.	Modificado	El grupo implementó la propuesta de utilizar un archivo de contadores, pero agregó una validación adicional. Se identificó que, si el archivo de datos de una entidad era eliminado mientras el archivo de contadores permanecía, los nuevos registros continuaban con el último valor almacenado (por ejemplo, iniciando en 12 en lugar de 1), generando una inconsistencia entre los datos y la numeración. Por ello, se implementó una lógica que, cuando el archivo de datos no existe o no puede recuperarse correctamente, reinicia el contador desde 1 , manteniendo la consistencia del sistema.
29/06/2026	Claude, DeepSeek	Genera la documentación JavaDoc para las	Generaron comentarios JavaDoc para las	Aceptado	La documentación generada

		clases y métodos de nuestra aplicación de gestión inmobiliaria. La documentación debe describir claramente la finalidad de cada clase, el comportamiento de los métodos, los parámetros, los valores de retorno y las excepciones que puedan lanzarse, siguiendo el estándar de JavaDoc.	clases y métodos del proyecto, incluyendo descripciones de las responsabilidades de cada clase, la funcionalidad de los métodos y la documentación de parámetros, valores de retorno y excepciones.		permitió mejorar la legibilidad y mantenibilidad del código, además de cumplir con las buenas prácticas de documentación exigidas para el desarrollo del proyecto.
29/06/2026	Claude, DeepSeek	Necesitamos implementar el método verDetalle para mostrar toda la información de un inmueble seleccionado dentro de la aplicación. Propón una implementación que recupere los datos del objeto y los presente de forma organizada en un cuadro de diálogo, utilizando la información ya almacenada en el sistema.	Generó una implementación que obtenía el inmueble seleccionado y construía un resumen con sus principales atributos para mostrarlo mediante un cuadro de diálogo.	Modificado	Se tomó como base la lógica propuesta, pero se adaptó el contenido mostrado para incluir todos los atributos definidos por el proyecto para cada inmueble en particular y mantener el formato visual utilizado en el resto de la interfaz.
29/06/2026	ChatGPT, Claude, Gemini	Genera una base de datos de prueba en formato Excel para una aplicación de gestión inmobiliaria. Incluye información coherente para inmuebles, inquilinos, alquileres, facturas y movimientos bancarios, de manera que pueda utilizarse para poblar los archivos .dat y realizar pruebas del sistema.	Generó una base de datos en formato Excel con registros de ejemplo organizados por cada entidad del sistema, incluyendo datos consistentes y relacionados entre sí.	Aceptado	La base de datos permitió cargar información de prueba de forma rápida y uniforme, facilitando las pruebas de funcionamiento de la aplicación sin necesidad de crear manualmente cada registro.
29/06/2026	Claude	Compara nuestro diagrama UML con la implementación final del proyecto en Java e identifica	Señaló ajustes en la ubicación de la composición entre VentanaPrincipal	Aceptado Parcialmente	Se realizaron las correcciones relacionadas con asociaciones, dependencias y

		inconsistencias relacionadas con asociaciones, dependencias, composición, representación de enumeraciones y clases de la interfaz. Indica únicamente los cambios necesarios para mantener coherencia entre el modelo y el código.	y los paneles, la dirección de algunas dependencias con InmuebleServicio , las asociaciones con Inmueble y la representación de las enumeraciones, además de aclarar que no era necesario incorporar clases propias de Swing al UML.		enumeraciones, mientras que se decidió no incluir clases de la biblioteca Swing (JFrame, JPanel y JDialog) por tratarse de detalles de implementación y no del modelo del sistem
--	--	--	---	--	---